KLE Technological University
Department of Computer Science & Engineering (AI)

Big Data Analytics (26ECAE417) · Course Project
Implementation of a Real-Time Application using Hadoop

Real-Time Breaking-News Detection from Wikipedia Traffic Using Hadoop

WikiTrend: Hadoop MapReduce · HDFS · YARN · Hive

Team member	USN
Member 1 Name	USN 1
Member 2 Name	USN 2
Member 3 Name	USN 3
Member 4 Name	USN 4

Under the guidance of: Mr. Pavankumar S / Ms. Soumya Patil
VII Semester, 2026-27

Contents

1. Project Title
2. Abstract
3. Introduction
4. Problem Statement
5. Objectives
6. Dataset Description
7. Software & Tools Used
8. Hadoop Architecture
9. Methodology
10. Implementation
11. Commands Used
12. Results / Output
13. Conclusion

1. Project Title

Real-Time Breaking-News Detection from Wikipedia Traffic Using Hadoop (short name: *WikiTrend*).

2. Abstract

When a major event happens, millions of people look it up on Wikipedia within minutes, so sudden surges in article traffic are an early, language-independent signal of breaking news. This project builds **WikiTrend**, a Hadoop-based system that ingests Wikimedia's public hourly pageview logs into **HDFS** and processes them with a chain of three **Java MapReduce** jobs. The jobs clean and aggregate the logs, learn every article's normal traffic level from a sliding six-hour baseline, flag statistically abnormal surges with a **z-score anomaly test**, and rank the trending articles of each hour. The system runs in two modes: a batch mode over the full dataset and a **real-time micro-batch mode** that processes each new hourly file as soon as it lands in HDFS. The results are analysed with **Hive** (external partitioned tables, an RCFile table, window functions and custom Java UDFs) and visualised as charts.

We evaluated the system on 30 hours of logs (7–8 September 2022, **5.6 GB, 183 million records, 327 language editions**), which include the death of Queen Elizabeth II. The full pipeline ran in 17 minutes on a single-node cluster and found 3,739 spikes. It detected the story at **12:00 UTC, five and a half hours before the official announcement**, from reader behaviour alone. It then tracked how the story spread to every major language edition. In real-time mode each new hour was fully processed in under three minutes, and the output matched the batch results exactly.

3. Introduction

Wikipedia is one of the most visited websites in the world, receiving roughly 20–35 million article views every hour. The Wikimedia Foundation publishes these counts openly: every hour a new file lists how many times each page of each project and language was viewed. A single hour is about 6.8 million lines and 200 MB uncompressed, so a day is about 4.5 GB. That is too much for a spreadsheet or a single script to analyse repeatedly, but well suited to the Hadoop ecosystem studied in this course.

Reader attention is a powerful real-time sensor. People turn to Wikipedia the moment they hear about an event, often before detailed news coverage exists, so a surge in views of an article is a sign that something newsworthy is happening. This works across all languages and topics, with no text analysis and no reliance on any news source. Detecting such surges is a **big-data stream-analytics problem**: the data arrives continuously (one file per hour), is large (Volume), is published every hour (Velocity), spans 300+ languages and many kinds of pages (Variety), and contains noise from automated traffic, page renames and navigation pages (Veracity).

WikiTrend implements this end-to-end on Hadoop. HDFS stores the raw logs and all intermediate results. MapReduce on YARN does the cleaning, aggregation, statistical anomaly detection and ranking. Hive provides an SQL layer for analysis. The project covers the course outcomes: big-data storage (HDFS, block storage, replication, CO2), distributed processing with MapReduce (CO3, CO4), stream processing concepts such as sources, sinks, sliding windows and stateful micro-batches (CO3), and analysis and visualisation with Hive (CO5).

4. Problem Statement

Given a continuous stream of hourly Wikipedia pageview logs (several GB per day, across 300+ language editions), detect **as soon as each hour's data arrives** which articles are receiving abnormally high attention compared with their own recent history, and rank them, so that breaking news and emerging trends can be identified automatically. The solution must store the data in HDFS, process it with Hadoop MapReduce, scale to the full volume of the logs, separate real surges from normal daily traffic swings, and make the results available for further analysis.

Scope: Wikipedia article pages only (desktop and mobile); hourly granularity (the publishing rate of the source); detection based on view counts only (no text analysis); evaluated on a historical window replayed as a live stream.

5. Objectives

1. Collect a public real-world dataset of at least 4 GB (achieved: 5.6 GB) and store it in HDFS.
2. Design and implement a multi-stage Hadoop MapReduce pipeline in Java to clean, aggregate and analyse it, using combiners, custom keys, partitioners, grouping comparators (secondary sort) and multiple outputs.

3. Implement a statistical anomaly detector (z-score against a sliding baseline window) that finds sudden surges in attention and ignores normal daily traffic patterns.
4. Run the pipeline both as a batch job and as a real-time micro-batch application that processes each new hourly file as it arrives.
5. Expose the results through Hive tables and queries, and visualise them.
6. Validate the system on a real event and measure its performance.

6. Dataset Description

Source: Wikimedia Foundation, *pageviews* dumps, `https://dumps.wikimedia.org/other/pageviews/` (public domain / CC0, no registration needed). One gzip file is published per hour, named `pageviews-YYYYMMDD-HH0000.gz`. Each line counts the views of one page in one project during that hour (user traffic only; known automated traffic is excluded by Wikimedia).

Table 6.1 Record format (space separated)

Field	Example	Meaning
domain_code	en, en.m, de.m, en.d	language + project; .m = mobile site; suffixes such as .d (Wiktionary) and .b (Wikibooks) mark other projects
page_title	Elizabeth_II	article title, spaces written as underscores
count_views	2221743	number of views in the hour
total_response_size	0	always 0 (kept for compatibility)

```
en Elizabeth_II 736731 0                  <- desktop
en.m Elizabeth_II 1485012 0               <- mobile (job 1 merges: 2,221,743 views at 18:00 UTC)
es.m Isabel_II_del_Reino_Unido 508242 0
en Special:Search 39399 0                 <- non-article page, dropped
en.d dog 6 0                              <- Wiktionary, dropped
```

Real lines from `pageviews-20220908-180000`.

Table 6.2 Dataset used in this project

Property	Value
Time window	2022-09-07 18:00 to 2022-09-08 23:00 UTC (30 consecutive hours)
Files	30 hourly files (1.5 GB compressed download)
Size in HDFS	**5,611,859,442 bytes (5.6 GB)**, uncompressed, 60 blocks of 128 MB, status HEALTHY
Records	183,201,631 lines (about 6.1 million per hour)
Coverage	327 Wikipedia language editions + other Wikimedia projects
Page views represented	654,348,373 article views after cleaning
Ground-truth event	death of Queen Elizabeth II: Palace statement on her health about 12:30 UTC, death announced 17:30 UTC on 8 Sep 2022

Why this window: it contains a globally known event with precisely documented timestamps, which lets us check whether the detector finds the right articles at the right time. The six hours of 7 September give the first hours of 8 September a full baseline. The same code runs unchanged on any other date via `DATA_START` in `scripts/env.sh`.

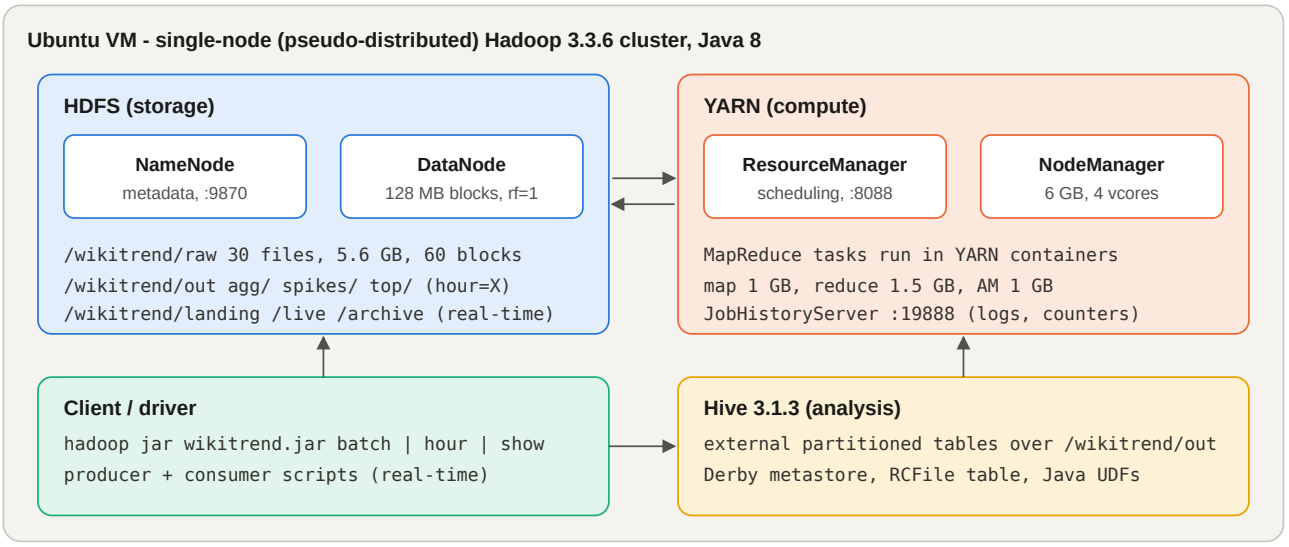
Data quality issues handled: other Wikimedia projects mixed into the same file; non-article pages (Special:Search, Talk:, File:, main pages, including localised namespaces); the same article counted separately for desktop and mobile; malformed lines; a long tail of millions of pages with 1–4 views; and pages created or renamed during an event (no history).

7. Software & Tools Used

Component	Version	Role in the project
Ubuntu Linux (VM)	64-bit	operating system of the single-node cluster
OpenJDK	1.8	runtime and compiler (Hadoop 3.3 and Hive 3.1 require Java 8)
Apache Hadoop – HDFS	3.3.6	distributed storage of raw logs, intermediate and final results
Apache Hadoop – YARN	3.3.6	resource management and scheduling of MapReduce containers
Apache Hadoop – MapReduce	3.3.6	the three processing jobs (Java API, `org.apache.hadoop.mapreduce`)
Apache Hive	3.1.3	SQL analysis on HDFS output; Derby metastore; RCFile; UDFs
Apache Maven	3.x	build of `wikitrend.jar`, JUnit 4 unit tests
Python 3 + matplotlib	3.x	charts from the Hive query results
Bash, curl, Git	–	automation scripts, data download, version control

8. Hadoop Architecture

The project runs on a **pseudo-distributed (single-node) Hadoop cluster**: every Hadoop daemon runs as a separate Java process on one Ubuntu VM and communicates over the network exactly as on a multi-node cluster, so the same jar runs unchanged on a real cluster.



HDFS

- **NameNode** keeps the namespace (directories, files, block locations); **DataNode** stores the 128 MB blocks. Each hourly file of about 190 MB is split into 2 blocks, so 30 files = 60 blocks.
- **Replication factor 1**, since there is a single DataNode (on a multi-node cluster the default of 3 gives fault tolerance).
- Files are stored **uncompressed** on purpose: gzip is not splittable, so a compressed hour would be read by one mapper; uncompressed, each block gets its own mapper (60 parallel map tasks).
- Results use Hive-style partition folders (`hour=2022-09-08T18`), so Hive can query them directly.

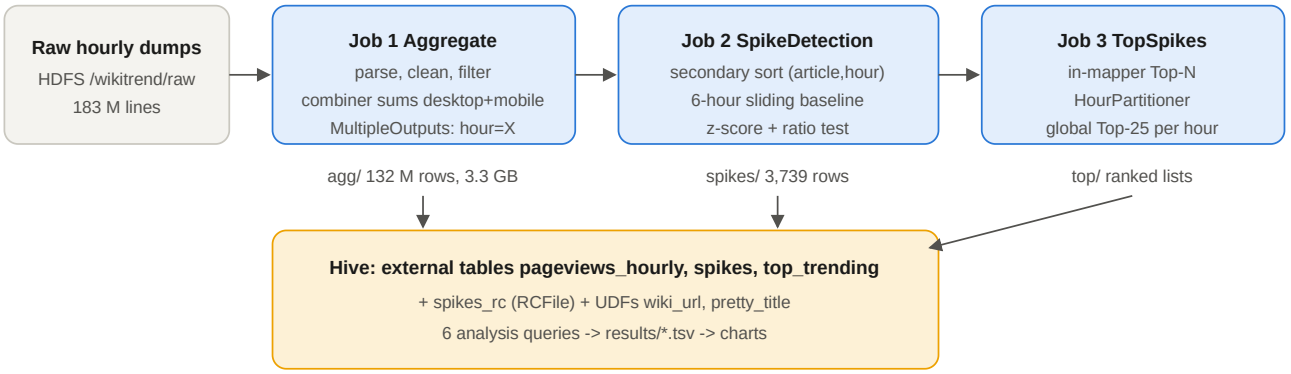
YARN and MapReduce

- The **ResourceManager** schedules applications; the **NodeManager** offers 6 GB / 4 vcores of containers. Each job gets an ApplicationMaster (1 GB) plus map (1 GB) and reduce (1.5 GB) containers.
- Map output is compressed during the shuffle; `io.sort.mb` is 256 MB to reduce spills.
- The **JobHistoryServer** keeps counters and logs of finished jobs (used for the numbers in this report).

Directory layout in HDFS

```
/wikitrend/raw/pageviews-20220907-180000 ... pageviews-20220908-230000     (input dataset, 5.6 GB)
/wikitrend/out/agg/hour=YYYY-MM-DDTHH/part-r-0000N                          (job 1)
/wikitrend/out/spikes/hour=.../part-r-0000N                                 (job 2)
/wikitrend/out/top/hour=.../part-r-0000N                                    (job 3)
/wikitrend/landing  /wikitrend/live/{agg,spikes,top}  /wikitrend/archive    (real-time mode)
/user/hive/warehouse/wikitrend.db/spikes_rc                                 (Hive RCFile table)
```

9. Methodology



9.1 Ingestion

The hourly dumps are downloaded from Wikimedia and decompressed directly into HDFS with `zcat file.gz | hdfs dfs -put - /wikitrend/raw/file`, without a temporary copy on local disk. The hour of each record is not stored inside the record; it is taken from the file name of the input split.

9.2 Job 1: cleaning and aggregation

- **Filter projects:** keep only Wikipedia (xx = desktop, `xx.m` = mobile); drop Wiktionary, Wikibooks, Commons, Wikidata and others (language code checked with a regular expression).
- **Filter pages:** drop non-article namespaces (Special:, Talk:, User:, File:, Category: ... and their German, Spanish, French, Italian, Russian, Japanese, Polish, Portuguese and Dutch forms) and main pages. Titles that merely contain a colon (e.g. *Star Wars: Andor*) are kept.
- **Merge** desktop and mobile views of the same article: key = `hour, lang, title`, value = views. A **combiner** sums on the map side, so duplicates are merged before the shuffle.
- **Output** with `MultipleOutputs` into one partition folder per hour.
- Hadoop **counters** record why each line was accepted or rejected (data-quality report).

9.3 Job 2: spike (anomaly) detection

For every article the reducer needs its views for all hours, **in time order**. This is done with a **secondary sort**: the map output key is the composite `(article, hour)`; a custom **partitioner** and a **grouping comparator** use only the article, so one `reduce()` call

receives the article's whole time series already sorted by hour. For each hour h to be scored, with v views and baseline B = views in the previous $W = 6$ hours:

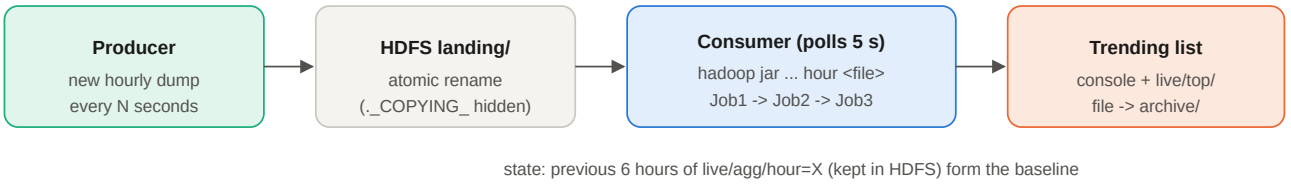
```
mean   = average(B)                      sigma = max( stddev(B), sqrt(mean + 1) )
z      = (v - mean) / sigma              ratio = v / (mean + 1)          excess = v - mean
spike  if  v >= 1,000  and  z >= 4  and  ratio >= 3
```

- The **z-score** measures how unusual this hour is for *this* article, so a page that always gets 1 M views is not flagged while a page that jumps from 300 to 12,000 is.
- The **sqrt(mean+1) floor** (Poisson noise) avoids a zero standard deviation for flat baselines (e.g. 0, 0, 0), which would make any visit infinitely surprising.
- The **ratio** and **minimum views** conditions remove statistically significant but practically tiny changes, as well as normal daily swings of popular pages.
- **Noise floor:** hourly counts below 5 are treated as 0 in the mapper. These rows are 84% of all rows but cannot influence a detection at the 1,000-view threshold, so not shuffling them makes job 2 much faster. Articles that never reach 1,000 views in any hour are skipped early in the reducer.
- Missing hours (no views) count as 0; at least 3 history hours are required before an hour is scored.

9.4 Job 3: ranking

Spikes are ranked by **excess views** (extra readers above the baseline), which measures the size of the story. Each mapper keeps only its local top 25 per hour in a bounded min-heap (**in-mapper combining**, emitted in `cleanup()`). A custom **HourPartitioner** spreads consecutive hours round-robin over the reducers, which merge the candidates into the global top 25 per hour.

9.5 Real-time micro-batch mode



Wikimedia publishes one file per hour, so the natural real-time unit is the hour, processed by **micro-batching** (the model used by Spark Streaming). A producer drops each new file into `/wikitrend/landing`. HDFS writes it under a temporary name and renames it atomically, so the consumer never sees a partial file. The consumer polls the folder every 5 seconds and runs the three jobs on the new hour only. The **state** of the stream (aggregated counts of the previous 6 hours) is kept in HDFS, so job 1 processes 1 file instead of 7. Job 2 is a **sliding-window, stateful** computation over the last 7 partitions. Each job writes to a private work folder whose partitions are moved into place only after success, so a failure or re-run never leaves half-written data (**idempotent** processing).

9.6 Analysis and visualisation

Hive **external partitioned tables** are declared over the three output folders (`MSCK REPAIR TABLE` registers the hour partitions). Spikes are also copied into a columnar **RCFile** table. Two Java **UDFs** (`wiki_url`, `pretty_title`) format the output. Six HiveQL queries (aggregations, a CTE + join, window functions) produce the tables and charts in Section 12.

10. Implementation

Table 10.1 Source code (Java 8, package com.wikitrend)

Class / file	Responsibility
WikiTrendDriver	entry point (Tool/ToolRunner); commands `batch`, `hour`, `show`; chains the jobs, publishes partitions atomically, prints counters
AggregationJob	Job 1: AggMapper, SumCombiner, AggReducer (MultipleOutputs, LazyOutputFormat)
PageviewParser	fast parsing and cleaning of one raw line; Quality enum used as counters
SpikeDetectionJob	Job 2: SpikeMapper (noise floor), SpikeReducer (sliding baseline window)
ArticleHourKey	composite WritableComparable key + ArticlePartitioner + ArticleGroupingComparator (secondary sort)
SpikeDetector	pure statistics (mean, sigma, z, ratio), unit-tested
TopSpikesJob	Job 3: TopMapper (in-mapper Top-N), HourPartitioner, TopReducer
HourUtil, Config	hour labels / epoch hours; tunable parameters (`-D wikitrend.*`)
WikiTrendTest	8 JUnit tests: parsing, filtering, hour labels, spike maths, zero-baseline edge case
hive/*.hql, WikiUrlUDF, PrettyTitleUDF	Hive tables, queries and UDFs
scripts/01..09_*.sh	installation, start-up, download, HDFS load, batch run, real-time producer/consumer, Hive

10.1 Job 1 mapper and cleaning

```
protected void map(LongWritable offset, Text line, Context context)
    throws IOException, InterruptedException {
  PageviewParser.Quality q = PageviewParser.parse(line.toString(), rec);
  context.getCounter(q).increment(1);
  if (q != PageviewParser.Quality.ACCEPTED) {
    return;
  }
```

```
String domain = line.substring(0, s1);
String lang;
if (domain.endsWith(".m")) {
  lang = domain.substring(0, domain.length() - 2);
} else {
  lang = domain;
}
if (!LANG.matcher(lang).matches()) {
  return Quality.NON_WIKIPEDIA_PROJECT;
}
```

10.2 Job 2: secondary sort configuration and the spike reducer

```
job.setMapperClass(SpikeMapper.class);
job.setReducerClass(SpikeReducer.class);
job.setMapOutputKeyClass(ArticleHourKey.class);
job.setMapOutputValueClass(LongWritable.class);
job.setPartitionerClass(ArticleHourKey.ArticlePartitioner.class);
job.setGroupingComparatorClass(ArticleHourKey.ArticleGroupingComparator.class);
```

```
protected void reduce(ArticleHourKey key, Iterable<LongWritable> values, Context context)
    throws IOException, InterruptedException {
  context.getCounter(Counters.ARTICLES_SEEN).increment(1);
  Arrays.fill(series, 0L);
  long peak = 0;
  // Secondary sort: values arrive in hour order, and Hadoop updates `key` in place while we
  // iterate, so key.getHour() is the hour of the current value.
  for (LongWritable v : values) {
    int idx = Arrays.binarySearch(available, key.getHour());
    if (idx >= 0) {
      series[idx] = v.get();
      peak = Math.max(peak, v.get());
    }
  }
  if (peak < minViews) {
    context.getCounter(Counters.ARTICLES_SKIPPED_LOW_PEAK).increment(1);
    return;   // can never be a spike: cheap early exit for the long tail
  }
  String article = key.getArticle().toString();

  for (long h : scoreHours) {
    int idx = Arrays.binarySearch(available, h);
    if (idx < 0) {
      continue;
    }
    int n = 0;
    for (int j = idx - 1; j >= 0 && available[j] >= h - window; j--) {
      baseline[n++] = series[j];
    }
    if (n < minHistory) {
      continue;
    }
    context.getCounter(Counters.HOURS_SCORED).increment(1);
    if (detector.score(series[idx], baseline, n, score)) {
      context.getCounter(Counters.SPIKES_FOUND).increment(1);
      line.set(article + '\t' + score.views + '\t' + fmt(score.mean) + '\t' + fmt(score.sigma)
          + '\t' + fmt(score.z) + '\t' + fmt(score.ratio) + '\t' + Math.round(score.excess));
      out.write(NullWritable.get(), line,
          HourUtil.PARTITION_PREFIX + HourUtil.fromEpochHour(h) + "/part");
```

10.3 Spike statistics

```
double sum = 0;
for (int i = 0; i < n; i++) {
  sum += baseline[i];
}
double mean = sum / n;
double sq = 0;
for (int i = 0; i < n; i++) {
  double d = baseline[i] - mean;
  sq += d * d;
}
double std = Math.sqrt(sq / n);
double sigma = Math.max(std, Math.sqrt(mean + 1));
double z = (views - mean) / sigma;
double ratio = views / (mean + 1);
if (z < zThreshold || ratio < minRatio) {
  return false;
}
out.views = views;
out.mean = mean;
out.sigma = sigma;
out.z = z;
out.ratio = ratio;
out.excess = views - mean;
```

10.4 Job 3 partitioner

```
public static class HourPartitioner extends Partitioner<Text, Text> {
  @Override
  public int getPartition(Text hour, Text value, int numPartitions) {
    return (int) (HourUtil.toEpochHour(hour.toString()) % numPartitions);
```

10.5 Hive table and UDF

```
CREATE EXTERNAL TABLE IF NOT EXISTS spikes (
  lang           STRING,
  title          STRING,
  views          BIGINT,
  baseline_mean  DOUBLE,
  sigma          DOUBLE,
  z_score        DOUBLE,
  ratio          DOUBLE,
  excess_views   BIGINT
)
PARTITIONED BY (hour STRING)
ROW FORMAT DELIMITED FIELDS TERMINATED BY '\t'
STORED AS TEXTFILE
LOCATION '${hivevar:OUT}/spikes';
```

```
public class WikiUrlUDF extends UDF {
  public String evaluate(String lang, String title) {
    if (lang == null || title == null) {
      return null;
    }
    return "https://" + lang + ".wikipedia.org/wiki/" + title;
```

10.6 Hive window-function query (Q4)

```
-- Q4. How the news spread across language editions: the #1 trending article of each language
-- in the hour after the announcement (window function ROW_NUMBER per language).
SELECT lang, article, views, baseline, excess_views
FROM (
  SELECT lang,
         pretty_title(title)                AS article,
         views,
         ROUND(baseline_mean)               AS baseline,
         excess_views,
         ROW_NUMBER() OVER (PARTITION BY lang ORDER BY excess_views DESC) AS rn,
         SUM(excess_views) OVER (PARTITION BY lang)                       AS lang_total
  FROM spikes
  WHERE hour = '2022-09-08T18'
) x
WHERE rn = 1
ORDER BY lang_total DESC
LIMIT 20;
```

11. Commands Used

```
# --- one-time setup: Java 8, Maven, Hadoop 3.3.6, configs, hdfs namenode -format, mvn package
scripts/01_install_hadoop.sh
# --- start daemons (no SSH needed)
hdfs --daemon start namenode;        hdfs --daemon start datanode
yarn --daemon start resourcemanager; yarn --daemon start nodemanager
mapred --daemon start historyserver; jps
# --- data ingestion (curl https://dumps.wikimedia.org/other/pageviews/2022/2022-09/...)
scripts/03_download_data.sh
hdfs dfs -mkdir -p /wikitrend/raw
zcat pageviews-20220908-180000.gz | hdfs dfs -put - /wikitrend/raw/pageviews-20220908-180000
hdfs dfs -ls -h /wikitrend/raw
hdfs dfs -du -s -h /wikitrend/raw
hdfs fsck /wikitrend/raw
# --- build and run MapReduce ---------------------------------------------------
mvn package
hadoop jar target/wikitrend.jar batch /wikitrend/raw /wikitrend/out
hadoop jar target/wikitrend.jar show /wikitrend/out 2022-09-08T18 15
hadoop jar target/wikitrend.jar -D wikitrend.z.threshold=6 batch /wikitrend/raw /wikitrend/out2
hdfs dfs -ls /wikitrend/out/top
hdfs dfs -cat /wikitrend/out/top/hour=2022-09-08T18/part-r-00000 | head
# --- real-time mode ------------------------------------------------------------
scripts/07_realtime_consumer.sh                     # polls: hdfs dfs -ls -C /wikitrend/landing
scripts/06_realtime_producer.sh 2022-09-08T12 2 60   # hdfs dfs -cp /wikitrend/raw/<hour>
/wikitrend/landing/
hadoop jar target/wikitrend.jar hour /wikitrend/landing/pageviews-20220908-120000 /wikitrend/live
hdfs dfs -mv /wikitrend/landing/pageviews-20220908-120000 /wikitrend/archive/
# --- Hive ----------------------------------------------------------------------
scripts/08_install_hive.sh       # schematool -dbType derby -initSchema
hive/build_udf.sh                # javac -cp hive-exec-3.1.3.jar ...; jar cf wikitrend-udf.jar
hive --hivevar OUT=/wikitrend/out -f hive/01_create_tables.hql
hive -S --hivevar UDF_JAR=hive/wikitrend-udf.jar -i hive/init.hql \
     -f hive/queries/q1_top_stories.hql
python3 viz/make_charts.py
# --- monitoring ----------------------------------------------------------------
yarn application -list
mapred job -status <job_id>
# Web UIs: NameNode :9870   ResourceManager :8088   JobHistory :19888
```

12. Results / Output

5.6 GB	183 M	17 min	3,739	< 3 min
dataset in HDFS (60 blocks)	records processed	full 3-job pipeline	spikes detected	per new hour (real-time)

12.1 Performance (single-node cluster, 4 vcores, 6 GB YARN memory)

Job	Input	Output	Time (s)
Job 1 Aggregate	30 raw files, 5.6 GB (60 splits)	132,293,753 article-hours, 3.3 GB	564
Job 2 SpikeDetection	120 partition files, 3.3 GB	3,739 spikes, 226 KB	291
Job 3 TopSpikes	spike partitions	top-25 per hour, 34 KB	172
Total (batch)			1027

Table 12.2 Hadoop counters of the batch run (data quality and filtering)

Counter	Value	Meaning
Raw lines read	183,201,631	all 30 hourly files
NON_WIKIPEDIA_PROJECT	11,638,900	Wiktionary, Wikibooks, Commons, Wikidata ... dropped
NON_ARTICLE_PAGE	5,710,212	Special:, Talk:, File:, main pages ... dropped
ACCEPTED	165,852,519	clean Wikipedia article records
VIEWS_ACCEPTED	654,348,373	page views represented by accepted records
ARTICLE_HOURS	132,293,753	(language, article, hour) rows after merging desktop + mobile
BELOW_NOISE_FLOOR	110,531,847	rows with < 5 views, not shuffled in job 2
ARTICLES_SEEN	2,877,917	distinct articles reaching the job-2 reducers
ARTICLES_SKIPPED_LOW_PEAK	2,873,859	never reach 1,000 views/hour: early exit
HOURS_SCORED	109,566	article-hours tested with the z-score
SPIKES_FOUND	3,739	anomalies detected

Job 1 reads the full 5.6 GB and dominates the runtime. The noise floor and the early exit cut Job 2's work from 132 M rows to about 22 M shuffled rows; only 4,058 of 2.9 M articles ever reach 1,000 views in an hour.

12.2 Detection timeline of the event

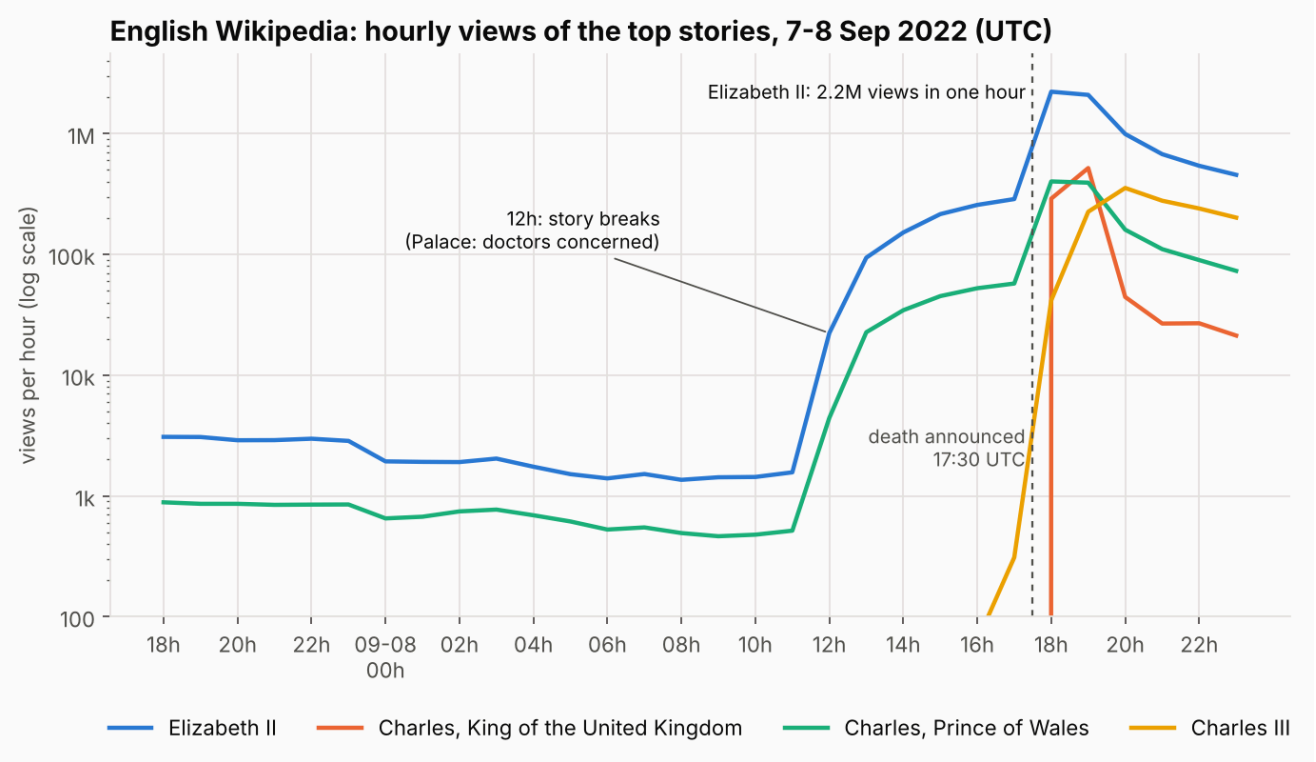


Figure 12.1 Hourly views of the biggest English stories (log scale). The surge starts at 12:00 UTC, when the Palace said doctors were concerned, five and a half hours before the announcement.

The detector's ranked output per hour (console output of the `show` command) tells the story:

```
--- hour 2022-09-08T11 UTC ---
#    lang  article                                views   baseline        z        x
1    fr    Futuroscope                             5921         12   448.65   461.38
2    ja    森口瑤子                                   4213         33   496.42   122.71
3    en    Peter Bone                              2224         39    33.57    55.60
4    ja    降旗康男                                   2030          5   526.32   348.00

--- hour 2022-09-08T12 UTC ---
#    lang  article                                views   baseline        z        x
1    ja    武田玲奈                                  22779         55  2132.24   410.43
2    en    Elizabeth II                           22311       1452   293.10    15.36
3    en    Operation London Bridge                14463         86   827.12   166.56
4    ru    AGM-114 Hellfire                        5360         93    34.90    57.02
5    en    List of longest-reigning monarchs       5163        220   166.09    23.36
6    fr    Élisabeth II                            4769        197   167.99    24.05

--- hour 2022-09-08T13 UTC ---
#    lang  article                                views   baseline        z        x
1    en    Elizabeth II                           93939       4937    11.45    19.02
2    en    Operation London Bridge                80819       2483    14.62    32.53
3    en    Balmoral Castle                        27816        781    19.42    35.59
4    en    Charles, Prince of Wales               22670       1155    14.69    19.61
5    es    Isabel II del Reino Unido              20781        472    29.81    43.98
6    en    Prince Philip, Duke of Edinburgh       19844       1021    12.92    19.42
```

- **11:00**: normal hour, small unrelated trends.
- **12:00**: *Elizabeth II* (z = 293), *Operation London Bridge* (the UK's plan for the Queen's death, z = 827), *List of longest-reigning monarchs* and the French article are flagged. **The story is detected 5.5 hours before the official announcement**.

- **13:00**: *Balmoral Castle* (her residence), *Charles, Prince of Wales*, the Spanish article.

```
--- hour 2022-09-08T18 UTC ---
#    lang  article                                  views   baseline       z       x
1    en    Elizabeth II                           2221743     171181    22.20   12.98
2    es    Isabel II del Reino Unido               604501      59105    13.37   10.23
3    en    Charles, Prince of Wales                401659      36083    20.09   11.13
4    en    Charles, King of the United Kingdom     289395          0 289395.00 289395.00
5    pl    Elżbieta II                             283174      13800    39.61   20.52
6    fr    Élisabeth II                            273288      22160    21.54   12.33
7    ru    Елизавета II                            301424      62930     4.24    4.79
8    it    Elisabetta II del Regno Unito           242125      26912    12.45    9.00
9    de    Elisabeth II.                           217071       9941    36.95   21.83
10   en    Prince Philip, Duke of Edinburgh        186374      26838    13.14    6.94
11   pt    Isabel II do Reino Unido                163194      13644    13.23   11.96
12   en    Prince William, Duke of Cambridge        98790       9997    17.30    9.88
```

At **18:00**, the hour after the announcement, 8 of the top 11 places are the Queen's article in English, Spanish, Polish, French, Russian, Italian, German and Portuguese. Rank 4, *Charles, King of the United Kingdom*, had **no traffic at all** in the previous six hours: it is a new page name created by editors as the news broke.

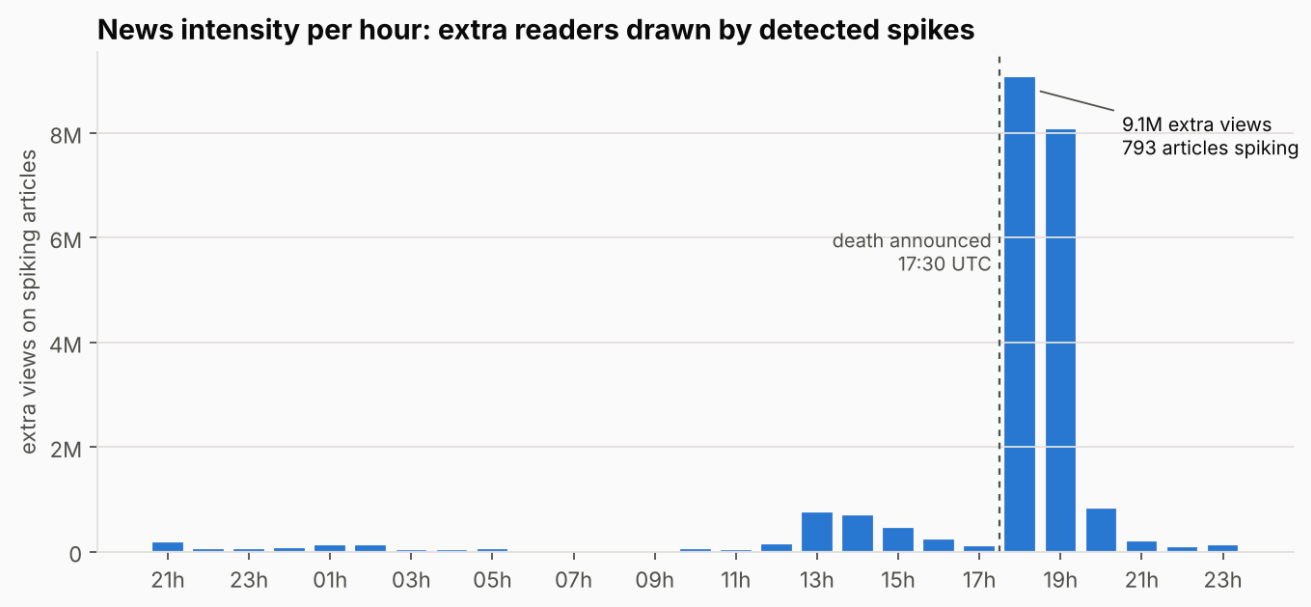


Figure 12.2 Total excess views of all detected spikes per hour: a quantitative measure of news intensity.

12.3 Hive analysis

Table 12.3 Q1: biggest stories of the period (ranked by largest hourly excess views)

lang	article	first	hours	peak views	max excess	max z
en	Elizabeth II	12h	3	2,221,743	2,050,562	293.1
es	Isabel II del Reino Unido	12h	4	604,501	545,396	35.2
en	Charles, King of the United Kingdom	18h	2	518,130	469,898	289395.0
en	Charles, Prince of Wales	12h	3	401,659	365,576	134.7
pl	Elżbieta II	12h	3	283,174	269,374	52.5
fr	Élisabeth II	12h	3	273,288	251,129	168.0
ru	Елизавета II	12h	5	301,424	238,494	50.0
en	Charles III	18h	2	225,315	218,295	401.3
it	Elisabetta II del Regno Unito	12h	4	242,125	215,213	43.6
de	Elisabeth II.	12h	3	217,071	207,131	37.0
en	Prince Philip, Duke of Edinburgh	12h	3	186,374	159,536	129.8
en	George VI	12h	4	191,127	157,600	81.6
pt	Isabel II do Reino Unido	13h	4	163,194	149,550	29.7
en	Operation London Bridge	12h	3	157,695	141,756	827.1
en	Death of Elizabeth II	18h	2	115,952	104,424	69166.0

Very large z values (e.g. 289,395) belong to pages with a zero baseline: with sigma floored at 1, z equals the view count. This is why spikes are ranked by excess views rather than by z (see also Table 12.5).

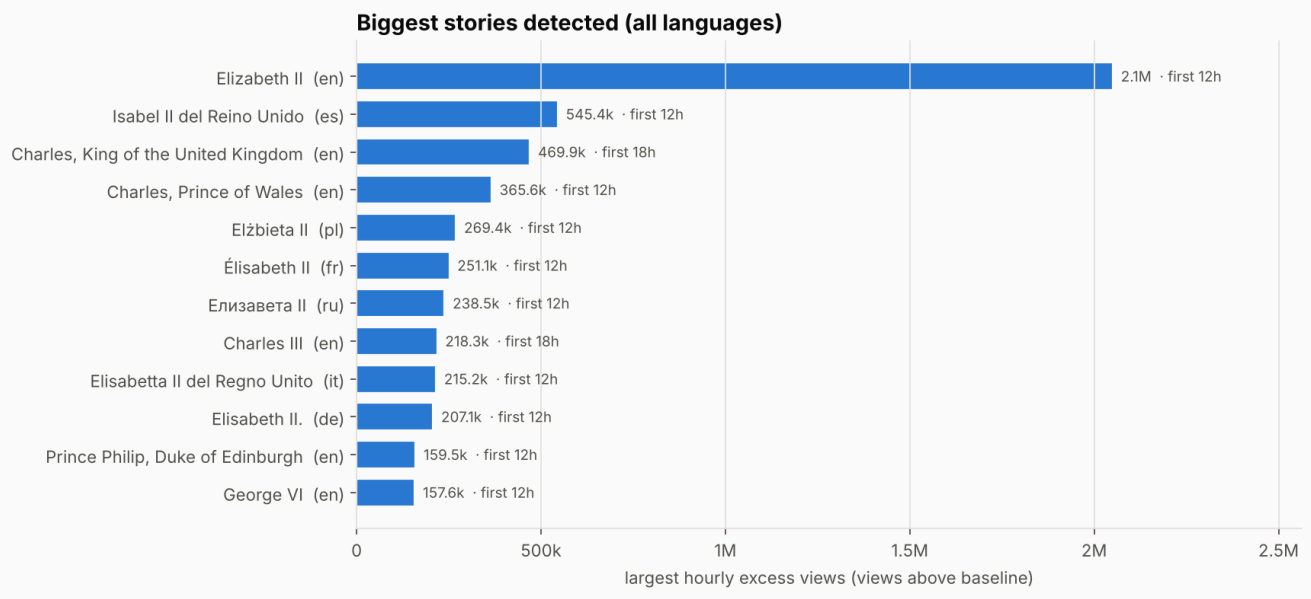


Figure 12.3 Biggest stories across all languages, with the hour of first detection.

Table 12.4 Q4: the number-one story in each language edition at 18:00 UTC (window function ROW_NUMBER)

lang	#1 trending article at 18:00 UTC	views	baseline	excess
en	Elizabeth II	2,221,743	171,181	2,050,562
es	Isabel II del Reino Unido	604,501	59,105	545,396
fr	Élisabeth II	273,288	22,160	251,129
pl	Elżbieta II	283,174	13,800	269,374
ru	Елизавета II	301,424	62,930	238,494
de	Elisabeth II.	217,071	9,941	207,131
it	Elisabetta II del Regno Unito	242,125	26,912	215,213
pt	Isabel II do Reino Unido	163,194	13,644	149,550
ar	إليزابيث الثانية	76,603	13,347	63,256
tr	II. Elizabeth	38,187	3,274	34,913
nl	Elizabeth II van het Verenigd Koninkrijk	44,580	4,533	40,047
cs	Alžběta II.	39,674	1,922	37,752
zh	伊丽莎白二世	49,615	7,191	42,424
ro	Elisabeta a II-a	42,602	1,381	41,221

Table 12.5 Q6 (RCFile table): articles born during the event: new or renamed pages with no previous traffic

hour	lang	article	views	baseline
18h	en	Charles, King of the United Kingdom	289,395	0.0
19h	ru	Карл III (король Великобритании)	70,161	0.0
18h	en	Death of Elizabeth II	69,166	0.0
19h	en	Prince William, Duke of Cornwall and Cambridge	47,382	0.0
19h	en	William, Duke of Cambridge	44,039	4.5
19h	fr	Charles III (roi du Royaume-Uni)	36,526	0.0
19h	en	Charles III, King of the United Kingdom	25,795	0.0
18h	de	Charles III. (Vereinigtes Königreich)	25,616	0.0
19h	de	Schmettow	25,187	0.0
18h	ru	Чарльз, король Великобритании	20,406	0.0
19h	pl	Karol III (król Wielkiej Brytanii)	18,139	0.0
19h	en	Prince William, Duke of Cornwall	16,609	1.7

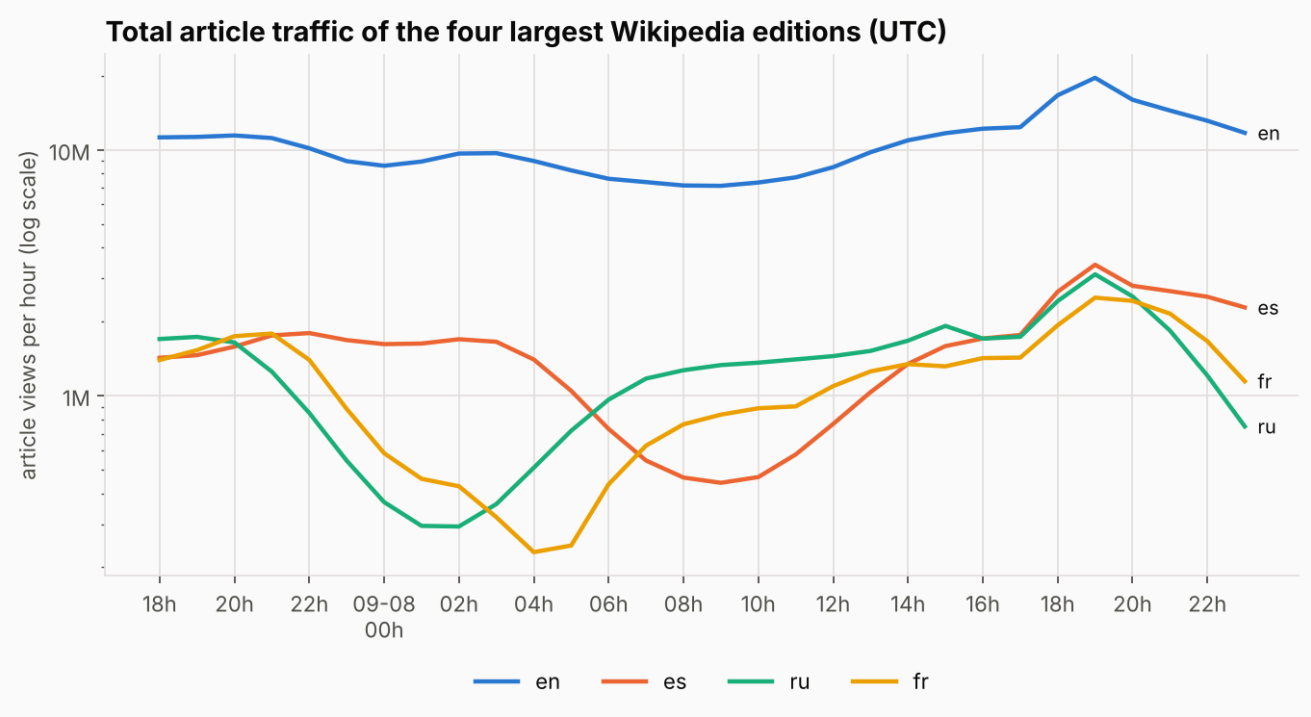


Figure 12.4 Total article traffic of the largest language editions: the daily rhythm of each region and the jump at 18:00-19:00 UTC.

12.4 Real-time mode

The producer published the 12:00 and 13:00 files one minute apart; the consumer detected each file and ran the three jobs on it. Console output (excerpt):

```
[19:05:57] >> Detected new file: /wikitrend/landing/pageviews-20220908-120000
[WikiTrend] New hour arrived: 2022-09-08T12  (file /wikitrend/landing/pageviews-20220908-120000)
[WikiTrend] Running WikiTrend-1-Aggregate ...
[WikiTrend] WikiTrend-1-Aggregate done in 51.9 s
      ACCEPTED                          6,201,402
      NON_ARTICLE_PAGE                    186,182
      NON_WIKIPEDIA_PROJECT               478,744
      ARTICLE_HOURS                     4,943,763
      VIEWS_ACCEPTED                   20,163,957
[WikiTrend] Baseline hours available: 6 of 6
[WikiTrend] Running WikiTrend-2-SpikeDetection ...
[WikiTrend] WikiTrend-2-SpikeDetection done in 95.4 s
      ARTICLES_SEEN                     1,531,340
      ARTICLES_SKIPPED_LOW_PEAK         1,530,870
      BELOW_NOISE_FLOOR                26,395,864
      HOURS_SCORED                            470
      SPIKES_FOUND                             54
[WikiTrend] Running WikiTrend-3-TopSpikes ...
[WikiTrend] WikiTrend-3-TopSpikes done in 26.0 s
[WikiTrend] Hour 2022-09-08T12 processed in 173.6 s
=== Trending on Wikipedia, hour 2022-09-08T12 UTC ===
#    lang  article                                          views   baseline        z        x
1    ja    武田玲奈                                           22779         55  2132.24   410.43
2    en    Elizabeth II                                     22311       1452   293.10    15.36
3    en    Operation London Bridge                          14463         86   827.12   166.56
4    ru    AGM-114 Hellfire                                  5360         93    34.90    57.02
5    en    List of longest-reigning monarchs                 5163        220   166.09    23.36
6    fr    Élisabeth II                                      4769        197   167.99    24.05
7    ja    村重杏奈                                             4266         20   548.62   199.97
8    ja    新山千春                                             4134         12   573.99   330.72
9    en    Charles, Prince of Wales                          4430        505   134.73     8.76
10   en    Prince Philip, Duke of Edinburgh                  4277        375   129.78    11.39
```

```
[19:09:02] >> Detected new file: /wikitrend/landing/pageviews-20220908-130000
[WikiTrend] New hour arrived: 2022-09-08T13  (file /wikitrend/landing/pageviews-20220908-130000)
[WikiTrend] Running WikiTrend-1-Aggregate ...
[WikiTrend] WikiTrend-1-Aggregate done in 52.3 s
      ACCEPTED                          6,472,548
      NON_ARTICLE_PAGE                    219,278
      NON_WIKIPEDIA_PROJECT               467,351
      ARTICLE_HOURS                     5,122,907
      VIEWS_ACCEPTED                   22,589,358
[WikiTrend] Baseline hours available: 6 of 6
[WikiTrend] Running WikiTrend-2-SpikeDetection ...
[WikiTrend] WikiTrend-2-SpikeDetection done in 90.8 s
      ARTICLES_SEEN                     1,601,217
      ARTICLES_SKIPPED_LOW_PEAK         1,600,564
      BELOW_NOISE_FLOOR                27,391,712
      HOURS_SCORED                            653
      SPIKES_FOUND                            200
[WikiTrend] Running WikiTrend-3-TopSpikes ...
[WikiTrend] WikiTrend-3-TopSpikes done in 25.7 s
[WikiTrend] Hour 2022-09-08T13 processed in 169.1 s
=== Trending on Wikipedia, hour 2022-09-08T13 UTC ===
#    lang  article                                          views   baseline        z        x
1    en    Elizabeth II                                     93939       4937    11.45    19.02
2    en    Operation London Bridge                          80819       2483    14.62    32.53
3    en    Balmoral Castle                                  27816        781    19.42    35.59
4    en    Charles, Prince of Wales                         22670       1155    14.69    19.61
5    es    Isabel II del Reino Unido                        20781        472    29.81    43.98
6    en    Prince Philip, Duke of Edinburgh                 19844       1021    12.92    19.42
7    en    List of longest-reigning monarchs                19398       1050     9.97    18.46
8    ja    駿河太郎                                           17595        117   577.91   149.11
9    ja    松本若菜                                           14992         94   802.86   157.81
10   ja    浅香航大                                           14014         92   730.83   151.50
```

Each hour was fully processed in under three minutes, well within the one-hour publishing interval of the source (latency under 5% of the interval). The ranked lists are **identical** to the batch run, which confirms that the incremental, stateful processing is correct.

> **Other events found without supervision** in the same 30 hours include the US Open tennis quarter-finals (*Jessica Pegula*, *Iga Świątek*, *Jannik Sinner*, *Carlos Alcaraz* at night UTC), the cricket Asia Cup (*Virat Kohli*, z = 43) and a monkeypox news item. They show that the detector is general and not tuned to one event.

13. Conclusion

WikiTrend shows that reader traffic on Wikipedia is an early, language-independent signal of breaking news, and that Hadoop can turn this large, noisy public log into a working real-time detection system. The project met all the guideline requirements: a real-time application built on Hadoop, a 5.6 GB dataset stored in HDFS, processing with Hadoop MapReduce, a working demonstration (batch and live), and this report.

Key outcomes:

- A three-stage MapReduce pipeline in Java that uses combiners, composite keys with secondary sort, custom partitioners and grouping comparators, MultipleOutputs, in-mapper Top-N and counters. It processes 183 M records in 17 minutes on a single node.
- A statistical anomaly detector (z-score on a sliding 6-hour baseline with a Poisson floor) that found the story **5.5 hours before the official announcement**, followed its spread

across eight-plus languages and also detected unrelated events (sport, health) without any tuning.
- A real-time micro-batch mode with state kept in HDFS and idempotent, atomic outputs: each new hour is processed in under 3 minutes, with results identical to the batch run.
- A Hive layer (external partitioned tables, RCFile, CTE/join, window functions, Java UDFs) and charts for analysis.

Limitations

- Hourly granularity is set by the source: an event is detected within the hour, not within minutes.
- When a story builds up over several hours (12:00–17:00), the baseline adapts, so the article is flagged at the start of the story and again at the much larger jump (18:00), but not in every hour in between.
- Page renames create "new" articles with no history (they are reported, correctly, as new pages).
- The cluster is a single node; the code is cluster-ready, but scaling was not measured on multiple machines.

Future work

- Group articles about the same event across languages using Wikidata IDs, to report one event instead of many articles.
- Use the minute-level Wikimedia EventStreams API with Spark Structured Streaming for lower latency.
- Seasonality-aware baselines (same hour last week) to further reduce false positives.
- Push alerts and a live dashboard on top of the `top_trending` table.

References

1. Wikimedia Foundation, Pageview complete dumps documentation, https://dumps.wikimedia.org/other/pageviews/readme.html
2. Apache Hadoop 3.3.6 documentation: HDFS architecture, MapReduce tutorial, YARN, https://hadoop.apache.org/docs/r3.3.6/
3. Apache Hive 3.1 documentation: LanguageManual DDL, RCFile, UDFs, https://cwiki.apache.org/confluence/display/Hive
4. Seema Acharya, Subhashini Chellappan, *Big Data and Analytics*, 2nd ed., Wiley India, 2022.
5. Gerard Maas, François Garillot, *Stream Processing with Apache Spark*, O'Reilly, 2019.
6. J. Lin, C. Dyer, *Data-Intensive Text Processing with MapReduce* (secondary sort, in-mapper combining), Morgan & Claypool, 2010.